

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 3

Roll no: 38 PRN No: 12413458 Name of Student: **Vedanti Raut**

Subject Name : Deep Learning

Problem Statement : Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm :

1. **Start**
2. **Import the required libraries** such as TensorFlow, NumPy, Matplotlib, and Scikit-learn.
3. **Load the Wine dataset** and separate it into input features X and target classes y.
4. **Split the dataset** into training and testing sets using an 80:20 ratio.
5. **Standardize the input features** using StandardScaler so that the features have a comparable scale.
6. **Create an MLP model** with:
 - Input layer with 13 features
 - Dense layer with 64 neurons and ReLU activation
 - Dense layer with 32 neurons and ReLU activation
 - Output layer with 3 neurons and Softmax activation
7. **Compile the model** using the Adam optimizer, categorical cross-entropy loss, and accuracy as the evaluation metric.
8. **Perform forward propagation** by passing the input data through the layers to obtain the predicted class probabilities.
9. **Calculate the loss** by comparing the predicted output with the actual class labels.
10. **Perform backpropagation** by calculating gradients of the loss with respect to the model weights and updating the weights using the Adam optimizer.
11. **Repeat forward and backward propagation** for the specified number of epochs until the model learns the classification patterns.
12. **Evaluate the trained model** on the test dataset and record its loss and accuracy.
13. **Analyze the effect of different learning rates** by training separate models using learning rates 0.1, 0.01, 0.001, and 0.0001.
14. **Record the test accuracy** obtained for each learning rate.
15. **Plot learning rate versus test accuracy** to compare their effect on model performance.
16. **Analyze the effect of the number of epochs** using the training and validation accuracy/loss curves.
17. **Compare the results** and identify the learning rate and number of epochs that provide better model performance.
18. **Stop.**

Code :

```
# 1. Import Libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
```

```
# 2. Load the wine dataset
wine = load_wine()

X = wine.data
y = wine.target

print("Input shape:", X.shape)
print("Target shape:", y.shape)
print("Class names:", wine.target_names)
```

Show hidden output

```
# 3. Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42, stratify=y)

print("Training samples:", X_train.shape)
print("Testing samples:", X_test.shape)
```

Show hidden output

```
# 4. Standardize the data
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("First training sample:")
print(X_train[0])
```

Show hidden output

```
# 5. Create the MLP Model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(13,)),

    tf.keras.layers.Dense(64, activation='relu'),

    tf.keras.layers.Dense(32, activation='relu'),

    tf.keras.layers.Dense(3, activation='softmax')
])
```

```
# 6. Compile the Model
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),loss='sparse_categorical_crossentropy',metrics=['accuracy'])
```

```
# 7. Train the Model
history = model.fit(X_train,y_train,epochs=20,validation_split=0.2,verbose=1)
```

```
# 9. Evaluate the Model
```

```
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print("Test Loss:", loss)
print("Test Accuracy:", accuracy)
```

Show hidden output

```
# 10. Plot Training Accuracy
```

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Training Accuracy', 'Validation Accuracy'])
plt.show()
```

Show hidden output

```
# 11. Plot Loss
```

```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Training Loss', 'Validation Loss'])

plt.show()
```

```
# 12.Run the learning-rate experiment
```

```
def create_model(learning_rate):

    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(13,)),
        tf.keras.layers.Dense(64, activation='relu'),
        tf.keras.layers.Dense(32, activation='relu'),
        tf.keras.layers.Dense(3, activation='softmax')
    ])

    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate), loss='sparse_categorical_crossentropy', metrics=['accuracy'])

    return model

learning_rates = [0.1, 0.01, 0.001, 0.0001]

lr_accuracies = []

for lr in learning_rates:

    print("Learning Rate:", lr)

    model = create_model(lr)

    model.fit(X_train, y_train, epochs=20, verbose=0)

    loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

    lr_accuracies.append(accuracy)

    print("Test Accuracy:", accuracy)
    print()
```

```
# 13. Plot learning rate vs accuracy

plt.plot(learning_rates, lr_accuracies, marker='o')

plt.xscale('log')

plt.xlabel('Learning Rate')
plt.ylabel('Test Accuracy')
plt.title('Effect of Learning Rate on Model Performance')

plt.show()
```

Output :

#2 Load the wine dataset

```
Input shape: (178, 13)
Target shape: (178,)
Class names: ['class_0' 'class_1' 'class_2']
```

#3 Split into training and testing data

```
Training samples: (142, 13)
Testing samples: (36, 13)
```

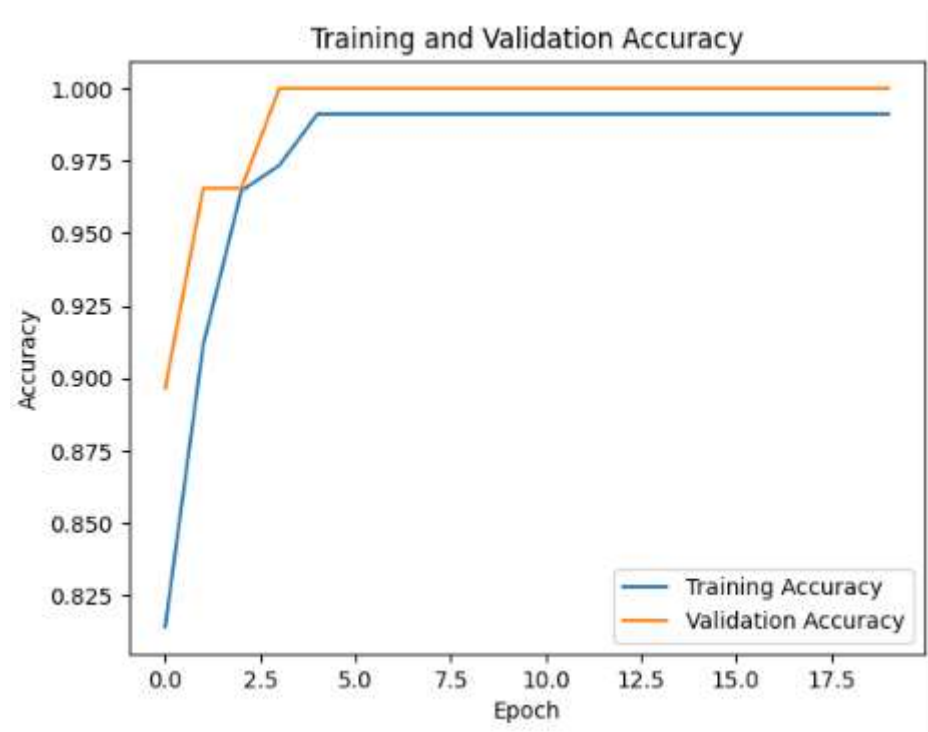
#4 Standardize the data

```
First training sample:
[ 0.38580089 -0.63787118  1.77666817 -1.22453161  0.69643032  0.52686525
 0.73229212 -0.1695489  -0.41578344 -0.16746725  0.62437819  0.2529082
 0.46772474]
```

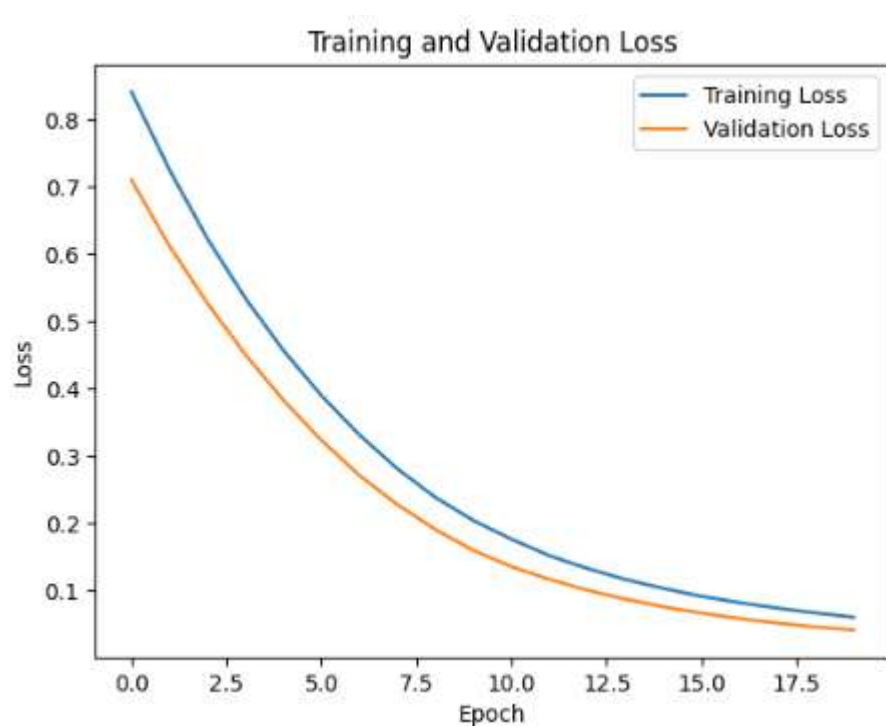
#7 Train the Model

```
Epoch 1/20
4/4 ----- 4s 405ms/step - accuracy: 0.8142 - loss: 0.8418 - val_accuracy: 0.8966 - val_loss: 0.7106
Epoch 2/20
4/4 ----- 0s 74ms/step - accuracy: 0.9115 - loss: 0.7257 - val_accuracy: 0.9655 - val_loss: 0.6124
Epoch 3/20
4/4 ----- 0s 55ms/step - accuracy: 0.9646 - loss: 0.6229 - val_accuracy: 0.9655 - val_loss: 0.5268
Epoch 4/20
4/4 ----- 0s 57ms/step - accuracy: 0.9735 - loss: 0.5341 - val_accuracy: 1.0000 - val_loss: 0.4503
Epoch 5/20
4/4 ----- 0s 46ms/step - accuracy: 0.9912 - loss: 0.4567 - val_accuracy: 1.0000 - val_loss: 0.3827
Epoch 6/20
4/4 ----- 0s 55ms/step - accuracy: 0.9912 - loss: 0.3894 - val_accuracy: 1.0000 - val_loss: 0.3228
Epoch 7/20
4/4 ----- 0s 61ms/step - accuracy: 0.9912 - loss: 0.3307 - val_accuracy: 1.0000 - val_loss: 0.2706
Epoch 8/20
4/4 ----- 0s 64ms/step - accuracy: 0.9912 - loss: 0.2804 - val_accuracy: 1.0000 - val_loss: 0.2270
Epoch 9/20
4/4 ----- 0s 81ms/step - accuracy: 0.9912 - loss: 0.2378 - val_accuracy: 1.0000 - val_loss: 0.1899
Epoch 10/20
4/4 ----- 0s 107ms/step - accuracy: 0.9912 - loss: 0.2035 - val_accuracy: 1.0000 - val_loss: 0.1590
Epoch 11/20
4/4 ----- 0s 54ms/step - accuracy: 0.9912 - loss: 0.1757 - val_accuracy: 1.0000 - val_loss: 0.1348
Epoch 12/20
4/4 ----- 0s 35ms/step - accuracy: 0.9912 - loss: 0.1511 - val_accuracy: 1.0000 - val_loss: 0.1158
Epoch 13/20
4/4 ----- 0s 30ms/step - accuracy: 0.9912 - loss: 0.1320 - val_accuracy: 1.0000 - val_loss: 0.0997
Epoch 14/20
4/4 ----- 0s 42ms/step - accuracy: 0.9912 - loss: 0.1157 - val_accuracy: 1.0000 - val_loss: 0.0863
Epoch 15/20
4/4 ----- 0s 39ms/step - accuracy: 0.9912 - loss: 0.1024 - val_accuracy: 1.0000 - val_loss: 0.0750
Epoch 16/20
4/4 ----- 0s 48ms/step - accuracy: 0.9912 - loss: 0.0906 - val_accuracy: 1.0000 - val_loss: 0.0656
Epoch 17/20
4/4 ----- 0s 43ms/step - accuracy: 0.9912 - loss: 0.0811 - val_accuracy: 1.0000 - val_loss: 0.0574
Epoch 18/20
4/4 ----- 0s 37ms/step - accuracy: 0.9912 - loss: 0.0730 - val_accuracy: 1.0000 - val_loss: 0.0508
Epoch 19/20
4/4 ----- 0s 32ms/step - accuracy: 0.9912 - loss: 0.0659 - val_accuracy: 1.0000 - val_loss: 0.0449
Epoch 20/20
4/4 ----- 0s 45ms/step - accuracy: 0.9912 - loss: 0.0595 - val_accuracy: 1.0000 - val_loss: 0.0403
```

#10 Plot Training accuracy



#11 Plot Loss



#12 Run the learning- rate experiment

```
Learning Rate: 0.1
Test Accuracy: 0.972222089767456

Learning Rate: 0.01
Test Accuracy: 1.0

Learning Rate: 0.001
Test Accuracy: 0.972222089767456

Learning Rate: 0.0001
Test Accuracy: 0.722222089767456
```

13. Plot learning rate vs accuracy

